

# Cohort Exercise - Sequence Diagram & Solution Modeling

## Learning Outcomes

By the end of this exercise, you should be able to:

1. Interpret system execution flow and design dynamic software logic using **Sequence Diagrams**.
2. Design software architecture using basic **Solution Class/Component Diagrams** using N-Tier (Layered) Architecture.
3. Understand the architectural differences between traditional Monolithic applications and modern Microservices.

**Total: 20 Marks**

---

## Task 1: Sequence Diagram Derivation (10 Marks)

*Transition Context: Now that you can read a sequence diagram, it is time to build one from scratch. This tests your ability to translate written business requirements into dynamic execution logic.*

Read the “Checkout and Payment” portion of the narrative from the campus food ordering system again: > “Upon selecting an item [...] the student needs to pay for the meal that she orders using a credit card. The system verifies the credit card payment. Upon successful payment, the system sends an email confirmation to the student for her order. The system sends an email and SMS notification to the food court stall regarding the new order. When the payment is unsuccessful, the system prompts the student to use a different payment method.”

Based on this narrative, **derive and draw a Sequence Diagram** representing the user checkout flow.

**Your diagram must include the following:**

1. **Identify the lifelines:** You must deduce and define at least five appropriate lifelines representing the standard layers of an N-Tier architecture. Include a UI (Boundary), Adapter (Boundary) a Controller, and the Models.
2. **Synchronous messages:** Use solid arrows (->) to represent method calls reflecting the narrative actions (e.g., verifying payment, updating status). **Ensure the Controller delegates business logic to the Service.**
3. **Return messages:** Use dashed arrows (-->) for returning data or boolean success/failure states back up the layers.

4. **Self-Calls:** Include at least one internal method call on a lifeline (e.g., a component calling its own internal notification or display logic).
  5. **Combined Fragments:** Use an **alt/else** block to accurately handle both the successful payment and unsuccessful payment scenarios described in the text.
- 

## Task 2: Solution Architecture - The Layered Monolithic Context (10 Marks)

*Transition Context: In this phase, we design the actual software code structure as a strictly layered Monolith. In this architecture, all code runs in the same memory space, but responsibilities are heavily segmented. Controllers handle web traffic, Services handle business rules, Repositories handle data access, and Domain Models hold state.*

Focusing precisely on the **Checkout Flow**, formulate the structural design for this layered web application.

### 1. Layered Responsibilities (5 Marks)

In this monolithic application, a checkout action requires the **Student** (buyer), the **Order** (cart), and the **PaymentGateway**.

*Explain the step-by-step flow of dependencies from top to bottom. How does data pass from the **CheckoutController**, down through the **CheckoutService**, to the **Domain Models**, and finally persist into the **Database** via the **Repository**? Justify why the Controller should not talk directly to the Database.*

### 2. Solution Component Visualization (5 Marks)

Draw a focused **Component or Class Diagram** showing a vertical architectural “slice” of the Checkout process.

- Your diagram must explicitly include elements representing the **UI**, **Controller**, **Service**, **Repository**, **Domain Models**, and the **Database**.
  - Draw navigability/dependency arrows showing the strict top-down flow of the layers (e.g., UI only talks to Controller, Controller only talks to Service).
- 

## Task 3: The Microservices Shift

*Context: Your layered monolithic application built in Task 2 has been highly successful. The university needs to scale the system to handle traffic jumps from 500 daily users to over 50,000 concurrent connections. The Chief Architect declares that the system will transition to a **Microservices Architecture**,*

*splitting the code into three completely independent physical environments: an Account Service, an Order Service, and a Payment Service.*

Discuss the following architectural implications with your cohort:

- **Severing Object References:** In a traditional monolith, the `CheckoutService` easily fetches a `Student` and an `Order` into shared RAM at the same time. If the `Order Microservice` and `Account (Student) Microservice` no longer share a database or server, why does this physical reference break the system? How must we change the `Order Domain Model` to logically represent this relationship across network boundaries?
  - **Distributed Workflow:** In your layered monolith, your `CheckoutService` simply called the `PaymentGateway` via a standard code import. In Microservices, the `PaymentService` runs on a completely separate server. How do these independent software stacks communicate to verify a credit card during checkout?
-